Practical 8: Sorting and Searching Algorithms

Aim

To study and implement Bubble Sort and Binary Search algorithms in Python.

Theory

Sorting algorithms arrange elements of a list into a defined order (typically ascending or descending) and are foundational to efficient searching and data processing. Bubble Sort is a simple comparison-based sorting algorithm that repeatedly steps through the list, compares each pair of adjacent elements, and swaps them if they are in the wrong order; after each full pass, the largest unsorted element 'bubbles up' to its correct position, giving a time complexity of O(n^2) in the average and worst case and O(n) in the best case (already sorted) with an early-exit optimization, and O(1) space complexity since sorting happens in place. Searching algorithms locate a target value within a collection. Linear Search checks elements sequentially (O(n)), while Binary Search is a much more efficient divide-and-conquer algorithm for finding an item in a sorted array: it repeatedly compares the target with the middle element and discards the half of the array that cannot contain the target, narrowing the search range until the element is found or the range becomes empty, achieving a time complexity of O(log n). Binary Search requires the input array to be sorted beforehand, which is why it is applied here after bubble_sort() has arranged the data.

Algorithm

1. Bubble Sort: for each pass, compare adjacent elements and swap them if the left is greater than the right; repeat until no swaps are needed.

2. Binary Search: set low and high pointers; compute the mid index; compare the middle element with the target; narrow the search range accordingly until found or the range is empty.

3. Sort the given array using bubble_sort().

4. Search for a target value in the sorted array using binary_search() and print its index.

Program

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

data = [64, 25, 12, 22, 11]
sorted_data = bubble_sort(data.copy())
print("Sorted Array:", sorted_data)
print("Index of 22:", binary_search(sorted_data, 22))
```

Output

```
Sorted Array: [11, 12, 22, 25, 64]
```

```
Index of 22: 2
```

Result / Conclusion

Bubble Sort and Binary Search were successfully implemented, sorting the array and locating the target element correctly.

Viva Questions

Q1. What is the time complexity of Bubble Sort in best, average, and worst cases?

Ans: Worst case (reverse sorted): $O(n^2)$ — every element must be swapped in every pass. Average case: $O(n^2)$. Best case (already sorted, with early-exit optimisation): O(n) — one pass with zero swaps detects sorted order. Space complexity: O(1) — in-place sorting with only a constant number of extra variables.

Q2. What is the prerequisite for Binary Search?

Ans: The array must be sorted in ascending (or descending, with the comparison adjusted) order before applying Binary Search. If the array is unsorted, Binary Search may miss the target entirely and return a wrong result. The cost of sorting (e.g., O(n log n) with merge sort) is worthwhile only if many searches will be performed on the same dataset.

Q3. How does Binary Search achieve O(log n) time complexity?

Ans: Binary Search halves the search space at each step. Starting with n elements: after 1 comparison n/2 remain, after 2 comparisons n/4 remain, and so on. After k comparisons, n/2^k elements remain. The search ends when n/2^k = 1, i.e., k = log■(n). So the maximum number of comparisons is log■(n), giving O(log n) time complexity.

Q4. Is Bubble Sort a stable sorting algorithm? What does 'stable' mean?

Ans: Yes, Bubble Sort is stable. A stable sorting algorithm preserves the relative order of equal elements — if two elements have the same key, the one that appeared first in the original array appears first in the sorted output. Bubble Sort is stable because it only swaps adjacent elements when strictly greater-than (not equal), so equal elements never swap past each other.

Q5. What are some more efficient alternatives to Bubble Sort?

Ans: Merge Sort: O(n log n) all cases, stable, uses O(n) extra space. Quick Sort: O(n log n) average, $O(n^2)$ worst case, in-place (O(log n) stack). Heap Sort: O(n log n) all cases, in-place, not stable. Python's built-in sorted() uses Timsort — O(n log n) worst, O(n) best, stable, hybrid of merge sort and insertion sort. Bubble Sort is mainly used for educational purposes, not in production.